

PayrollGuard AI — Updated MVP Roadmap (SIN 2026 Edition)

Mission: Build a working, fully local demo of PayrollGuard AI good enough to win a top-15 preselection slot in the National Competition for the Best ICT Project — without spending time on anything that doesn't move that needle.

The one rule that governs every decision below:

The written submission gets you into the room. The engineering only matters if you're in the room. Don't let the build outgrow the pitch.

Registration closes **22 July 2026, 3:30 PM**. Preselection results: **24 July, 12 noon**. Everything in this document is sequenced backward from that date.

0. Priority Order (read this first)

1. **The written project document** — this is what gets you to top 15. Nothing else matters if this is weak.
2. **A working core pipeline** (upload → detect → score → report) — proves technical feasibility, the thing the document claims.
3. **The WOW demo** — only relevant *after* you're preselected, for the bootcamp/pitch phase (27–30 July).
4. **Everything else** (PDF export, settings, polish) — cut unless time remains.

If you're behind schedule at any point, cut from the bottom of this list, never the top.

1. Tech Stack (final, trimmed)

Layer	Tool	Why
Frontend	Next.js 14 + Tailwind + ShadCN	Fast to build, looks credible
Charts	Recharts	Minimal setup
Graph viz	React Flow	Fraud network visual
Backend	FastAPI (Python)	Async, plays well with data/ML libs
Data processing	Pandas	Vectorized, handles 14k+ rows instantly
Duplicate/exact matching	Pandas groupby	No library needed — this is just SQL-style logic
Fuzzy name matching	RapidFuzz + blocking (see §3)	Fast even at scale
Fraud network discovery	NetworkX (connected	Standard graph algorithm, well-

Layer	Tool	Why
	components)	documented, explainable
Salary anomaly detection	Isolation Forest (scikit-learn) + grade/duty-post expected-salary rule	ML catches the unknown; the rule catches the explainable
Local LLM	Ollama running Llama 3.1 8B (or similar 8B-class model)	Final report generation ONLY — fully offline
Database	PostgreSQL (or SQLite for demo simplicity)	SQLite is fine for a single-laptop demo; don't over-engineer
Hosting	Docker Compose, local machine	No cloud dependency — this IS the pitch

Cut from original plan: Redis, react-pdf/html2canvas export, settings page, multi-user auth, Render/Railway hosting. None of these affect scoring or the demo.

2. Data Schema (matched to real Cameroonian payroll structure)

Employee Registry CSV

```
matricule, full_name, national_id, phone, ministry, department,
grade, duty_post_code, category, class_echelon, hire_date, location
```

Payroll CSV

```
matricule, employee_name, base_salary, allowance_codes, total_salary,
bank_name, bank_account, payment_date
```

Why matricule, not employee_id: this is the real identifier used in SIGIPES/AIGLES. Using it signals domain understanding to a jury that knows the system.

Demo allowance codes to include (don't go beyond these — full code table is 100+ entries and not worth your time): housing, duty, representation, transport, family.

3. The Algorithm Layer (deterministic — no AI here)

This is the core insight to hold onto: **almost nothing in detection should touch an LLM.**

Detection needs to be fast, explainable, and defensible — exactly what classical algorithms give you, and exactly what AI usually doesn't.

Step-by-step pipeline

1. **Ingest** — Pandas reads both CSVs, normalizes text (trim, uppercase names for matching, validate columns).

2. **Exact-match detection** — `groupby('national_id').filter(count > 1)`, same for `bank_account`, same for `matricule`. This is near-instant regardless of file size.
 3. **Blocked fuzzy name matching** — Don't compare every name to every other name ($O(n^2)$, slow at scale). Instead:
 - Bucket names by first letter of surname (or a phonetic code like Soundex).
 - Run RapidFuzz only *within* each bucket.
 - Flag pairs $\geq 85\%$ similarity.
 - This turns a 98-million-comparison problem (at 14,000 rows) into something that completes in well under a second.
 4. **Missing-record detection** — Vectorized set difference: `payroll_matricules - registry_matricules = potential ghost workers`.
 5. **Salary anomaly detection** — Two layers:
 - **Isolation Forest** on salary distribution, for unknown/unexpected patterns.
 - **Expected-salary rule**: compare `total_salary` against what the `grade/duty_post_code/class_echelon` combination should pay (your demo emoluments lookup table). This is the more *explainable* signal — "this salary should be X, it is Y" beats "the model says this is statistically weird" in front of a jury or auditor.
 6. **Fraud network construction** — Build a graph: nodes = employees, edges = shared bank account / shared national ID / fuzzy-matched name. Run NetworkX connected components. Clusters of 2+ = a fraud network.
 7. **Risk scoring** — Weighted sum per employee (duplicate ID +30, shared account +25, missing record +20, salary anomaly +15, network membership +10), capped at 100, mapped to Low/Moderate/High/Critical.
 8. **AI report generation (last step, local LLM only)** — Feed the structured JSON output of steps 2–7 into the prompt template. The model writes prose. It never sees raw rows, never makes detection decisions, never touches anything you can't already prove with steps 1–7 alone. This is the entire job of AI in this system, and you should say that explicitly in your pitch — it's a strength, not a limitation.
-

4. Performance & Latency Plan

- **14,000 rows is not "big data."** Pandas loads it in well under a second. Don't build chunked/streaming ingestion — that solves a problem you don't have.
- **The only real bottleneck is naive fuzzy matching.** Fix it with blocking (§3, step 3). This alone keeps your pipeline fast at any realistic Cameroonian ministry scale.
- **Run analysis as a background task**, not a blocking HTTP call (FastAPI BackgroundTasks is enough — no need for Celery/Redis). Frontend polls a `/status` endpoint or shows a progress indicator. This matters more for demo psychology than raw

speed: a visible "Cross-referencing 14,000 bank accounts..." beats a frozen screen.

- **Local LLM is your one genuine latency risk.** Mitigate by: generating the report last (after all other results are already shown), streaming tokens if your Ollama setup supports it, and testing your actual demo hardware early — if an 8B model is too slow, drop to a smaller one rather than abandoning local-only architecture (it's now core to your pitch).
-

5. Timeline (3 weeks to registration deadline — 22 July)

This timeline assumes you are also juggling ADTC, the AI PC companion project, and client work. Budget realistically — if something has to give in a given week, it should not be the document.

Week 1 (Days 1–7): Document First, Skeleton Second

- **Days 1–2 — Write the project document.** This is the actual top priority. Cover all 7 scoring criteria explicitly (see §6 checklist). Position PayrollGuard as complementary to AIGLES, not competing with it. Get a second pair of eyes on it.
- **Days 3–4 — Repo setup.** Next.js + FastAPI skeletons, Docker Compose, SQLite/Postgres, basic upload endpoint.
- **Days 5–7 — Generate your demo dataset now.** 14,000-row synthetic Ministry-of-Education-style dataset (matching your own concept doc's scenario), seeded with known fraud networks, ghost workers, duplicate IDs. Test it against even a rough version of the pipeline immediately — don't wait until week 3 to discover scale problems.

Week 2 (Days 8–14): Core Pipeline

- Days 8–9: Exact-match detection (duplicate ID, bank account) + missing-record detection.
- Days 10–11: Blocked fuzzy name matching. Benchmark against your 14k-row dataset — this is the step most likely to surprise you, test it early.
- Days 12–13: Salary anomaly detection (Isolation Forest + expected-salary rule) + fraud network graph (NetworkX).
- Day 14: Risk scoring engine. By end of week 2 you should have a backend that takes two CSVs and returns full structured JSON results — no UI needed yet to validate this.

Week 3 (Days 15–21): UI, Local LLM, Submission Polish

- Days 15–16: Dashboard page (cards, charts) + employee risk table, wired to real backend output.
- Days 17–18: Fraud network graph visualization (React Flow) — this is your single most impressive visual, prioritize it over anything else in the UI.
- Day 19: Ollama integration + report prompt tuning. Test on your real demo data, not lorem ipsum.
- Day 20: End-to-end smoke test. Time the full pipeline on your demo dataset — confirm it's under your own 20-second target.

- **Day 21 (by 22 July, 3:30 PM) — Submit.** Document + whatever working prototype you have is ready, even if some polish items are unfinished. A working core beats a fully-finished but late submission.

If You Make Top 15 (24 July onward)

Only at this point do the "WOW demo script," pitch deck slides, and any remaining polish (PDF export, settings page) become worth your time. Don't build for the pitch stage until you know you're pitching.

6. Document Checklist — All 7 Scoring Criteria (this matters more than code)

Use this as a literal checklist before submitting. Each criterion below is worth real points — don't let the jury have to infer any of these, state them directly.

- ☐ **Relevance (20 pts)** — Open with the National Assembly ghost-worker case (April 2026) and the 315-dead-but-still-paid detail. Make the problem concrete and Cameroonian, not generic.
- ☐ **Innovation (15 pts)** — Be explicit about what's novel: continuous, fast, independent forensic auditing layer vs. periodic manual headcounts; fraud *network* discovery (not just individual flags) as the key differentiator.
- ☐ **Technical feasibility (15 pts)** — Reference your actual working prototype, your real algorithm choices (not vague "AI"), and your realistic data schema (matricule-based, matching real SIGIPES/AIGLES structure).
- ☐ **Socio-economic impact (15 pts)** — Use the real numbers: CFA 30 billion/year saved by Coppe since 2019, FCFA 114 billion lost to corruption in 2023 (CONAC). Tie your tool to recovering a measurable slice of that.
- ☐ **Business model (10 pts)** — Concrete pricing, concrete buyer (ministries, NGOs, large private payrolls), a believable first-12-months path. Don't leave this vague.
- ☐ **AI use (10 pts)** — Be precise and honest: deterministic algorithms do detection, AI does report-writing only. This reads as more credible than "AI finds the fraud," not less impressive.
- ☐ **Digital patriotism & cybersecurity (15 pts)** — This is the one most competitors in your sector will underweight. State explicitly: fully local, on-premise, no cloud dependency, no government data ever leaves Cameroonian infrastructure, positioned as complementary to (not competing with) AIGLES.

Explicit AIGLES positioning line to include somewhere in the document:

"AIGLES is Cameroon's system of record for civil service payroll. PayrollGuard AI is an independent, lightweight forensic audit layer that can analyze exports from AIGLES, legacy SIGIPES, or any private payroll system — without requiring integration access, procurement cycles, or months of onboarding."

7. What to Deliberately NOT Build (for now)

- Multi-user auth / role-based access — irrelevant to scoring or demo.
 - PDF export — nice-to-have, not demo-critical; mention as a roadmap item instead.
 - Settings page — cut entirely.
 - Live SIGIPES/AIGLES API integration — explicitly out of scope, say so in the document as a "post-MVP" item, it shows maturity not weakness.
 - Redis/Celery/chunked file streaming — solving a scale problem you don't have at MVP size.
-

8. One-Sentence Summary to Keep on a Sticky Note

The document wins you the seat at the table. The demo only matters once you're sitting in it. Build exactly enough engineering to make the document true, and not one feature more.